

LISTA DE EXERCÍCIOS 1.2

1. Interrupções de *timer* poderiam ser usadas para computar o tempo corrente. Ofereça uma descrição de como isto poderia ser alcançado?

Resposta: Um programa poderia usar a seguinte abordagem para computar o tempo corrente usando interrupções de *timer*. O programa poderia ajustar um *timer* para algum momento futuro e simplesmente dormir (*sleep*). Quando ele é acordado pela interrupção, ele poderia atualizar seu estado local, o qual ele está usando para detectar o número de interrupções recebidas até então. Ele poderia repetir este processo de ajuste de *timer* continuamente e atualizar seu estado local quando as interrupções são efetivamente acionadas.

Exemplo: em um script *relogio.sh*

```
#!/bin/sh
counter=0
while true; do
    clear >$(tty)
    counter=$((counter+1))
    echo "Timer: $counter s"
    sleep 1
done
```

2. Considerando a hierarquia de dispositivos de armazenamento, explique por que os sistemas computacionais procuram um equilíbrio entre o uso dos tipos de memória existentes. Por que não utilizar a memória mais rápida para compor todo um sistema?

Resposta: Quanto mais rápida a memória, na hierarquia de memória, maior o seu custo. Considerando, por exemplo o segmento de computadores PC (notebooks, desktops etc), existe uma necessidade em se balancear o valor agregado do equipamento ao poder aquisitivo do usuário pretendido. Dessa forma, opta-se pelo equilíbrio entre o uso de tipos de memória mais rápidos em pequena quantidade e memórias mais lentas (ex. disco) os quais são mais baratos e permitem alcançar um valor médio proporcional ao poder de compra do usuário final. Deve-se observar também que, a proporção entre os tipos de memória também está relacionada ao desempenho mínimo aceitável para o usuário final deste nicho de mercado. Esta mesma observação pode ser ajustada para outros segmentos (ex. *Mainframes*, computadores paralelos, *workstations*, etc).

3. Explique em linhas gerais o funcionamento de DMA (Direct Memory Access).

Resposta: DMA é um dispositivo que funciona como um intermediário entre a CPU e dispositivos. O princípio fundamental é que o fluxo de interrupções gerados por dispositivos que transferem dados em blocos (ex. disco) podem sobrecarregar o barramento ao tentar notificar byte a byte, a transferência de dados entre dispositivo de I/O e memória. Assim, todo o trâmite intermediário (i.e. as interrupções intermediárias) seriam reportados à DMA e não a CPU. A CPU comunica no início de uma operação de I/O os dados desta para a DMA e o gerenciamento de DMA se encarrega de todo o trâmite de tratamento de interrupções intermediário, só reportando o término da operação de I/O em caso de sucesso ou erro em caso de falha.

4. Explique a diferença entre os métodos de E/S síncrono e assíncrono.

Resposta: no método de E/S síncrono, o processo obrigatoriamente aguarda o término de E/S para só então dar continuidade a suas atividades (E/S bloqueante). No método de E/S assíncrono, o processo realiza a solicitação de E/S, porém, ao contrário do método síncrono, o processo tem condições de dar continuidade a suas outras atividades, as quais independem desta E/S. Via de regra, o método assíncrono está relacionado com a implementação de um arcabouço de paralelismo na implementação de processo do SO (i.e. uso de threads).

5. Explique o princípio de *caching* aplicado a estrutura de um SO.

Resposta: o princípio de *caching* está relacionado com a cópia de uma informação solicitada de um armazenamento mais lento para um armazenamento mais rápido. Esse princípio ocorre em diversos níveis do SO, como por exemplo, na cópia de informação de processo (PCB) do disco para a memória principal e na cópia de endereço de referência de tabela de processo para o cache da CPU (TLB).

6. Defina multiprocessamento simétrico (SMP) e assimétrico (AMP).

Resposta: Em multiprocessamento simétrico, existe a implementação de múltiplas CPUs, onde cada uma possui a mesma arquitetura. Estas compartilham o espaço de memória (ou pelo menos parte dele). Neste cenário, normalmente um SO é usado e existe uma instância executando em todas as CPUs, dividindo o trabalho entre elas. Para tanto, algum tipo de comunicação entre CPUs é necessário (ex. memória compartilhada). Em multiprocessamento assimétrico, também existe a implementação de múltiplas CPUs, porém, nesse caso, cada uma pode possuir uma arquitetura diferente (pode ser a mesma também). Cada CPU possui seu próprio espaço de endereçamento, no qual, cada uma pode ou não executar um SO (não necessariamente o mesmo). Nesse caso, também é oferecido algum tipo de comunicação entre CPUs.